

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

Centralized Access Point for Information System Integration Problems
in Large Enterprises

by

Mai Minh Hai – MSE501

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

MINISTRY OF EDUCATION AND TRAINING
FPT UNIVERSITY

Centralized Access Point for Information System Integration Problems
in Large Enterprises

by

Mai Minh Hai – MSE501

A thesis submitted in conformity with the requirements
for the degree of Master of Software Engineering

Supervisor:

1. Dr.Vu Thu Diep

Centralized Access Point for Information System Integration Problems in Large Enterprises

Mai Minh Hai

Degree Master of Software Engineering

FPT University

2020

Abstract

The role of information systems is extremely important for a business, especially those operating in the field of technology. The information system works with people, information technology and processes to accomplish business objectives. In the development process of large companies, the member units create many applications to serve their purposes. Increasing the number of applications or portals consumes employees' time, and they may eventually be unable to keep track of all the necessary or useful information. That is why it is necessary to build a centralized and intelligent information access point for large organizations. The thesis presents such a solution for the largest software company in Vietnam, the FPT software company. The work describes the architectures and technologies applied to build an access point that brings whole new experiences to all employees. By integrating smart features based on artificial intelligence, the work contributed to enhancing performance and quality of work for 28,000 internal users. The approaches and results of the thesis are completely applicable to similar large company models.

Table of Contents

Table of Contents

Table of Contents	2
List of Tables	3
List of Figures	4
Chapter 1	1
Introduction	1
1- Introduction	1
Chapter 2 System Architecture and Requirements	3
2. System Architecture and Requirement	3
2.1 System Architecture	3
2.2 System requirements	3
Chapter 3	5
System design and implementation	5
3. System Design and Implementation	5
3.1 Servers	5
3.2 Monolithic and Microservice applications	5
3.2.1 Monolithic	5
3.2.2 Microservice	6
3.3 UniGate System	10
3.4 AI Bot	11
3.4.1 Why are chatbots important	11
3.4.2 Types of Conversational Bots	11
3.4.3 Benefits of Conversational Bots	11
3.4.4 RASA Platform	12
3.5 UniCorn Bot	16
3.6 Enterprise Service Bus	17
3.6.1 ESB and SOA	17
3.6.2 Benefits	17
3.6.3 ESB vs. Microservices	18
3.6.4 Mule ESB	22
3.7 Data Consolidation & Data Warehouse	23
3.7.1 Data Consolidation	23
3.7.2 Oracle Data Integrator	24
3.7.3 Data Warehouse	27
3.8 Web Application	29
Chapter 4	31
Result	31
4. Result	31
Chapter 5	33
Conclusion and Perspective	33
5. Conclusion and Perspective	33
References	33

List of Tables

Table 1. 10

Table 2. 11

List of Figures

Figure 1.	9
Figure 2.	12
Figure 3.	13
Figure 4.	16
Figure 5.	18
Figure 6.	22
Figure 7.	25
Figure 8.	29
Figure 9.	36
Figure 10.	37
Figure 11.	37
Figure 12.	37
Figure 13.	38
Figure 14.	38

Chapter 1

Introduction

1- Introduction

The process of developing management information systems usually forms and develops step by step along with the development of enterprises. The pressures for implementing an enterprise information system could be globalization, overcapacity, and reengineering, dealing with changing environments, etc. Enterprise information systems are implemented for the purposes of the integration of business processes, cycle time reduction, faster information transactions, Improvements in financial management, making tacit process knowledge explicit.

For large companies, as the number of applications increases, users spend more time to remember and perform the login. In addition, information on system usage is not well managed if the applications are independent.

This study examines the applications built on the information systems of Vietnam's largest software company, the FPT Software [1]. The company has about 40 on premise applications being used by more than 28,000 programmers, employees worldwide. These applications cover many key business areas, helping support the day-to-day management and administration of the company such as Human Resources, Sales & Marketing, Accounting & Finance, Delivery, Administrators, etc. Applications are developed using different platforms and technologies, integrated through the Application Programming Interface (API).

While researching solutions to build a centralized and intelligent access point for FPT Software company, several related studies were found. Some research related to service integration platforms by utilizing Service Oriented Architecture (SOA) [2,3], Enterprise Service Bus (ESB) [4,5], meta-analysis [6]. Rini Priantari and al. in [6] see that the use of different and non-standard information technology in an organization will produce silo data that affects things like, non-uniform data formats, data duplication, and information. To overcome this problem, the different information technologies integration is carried out. The study gives a information technology (IT) integration metanalysis were carried out in an organization to gain an understanding of the architecture or layers, technology and integration platform features. In a study conducted by Singh [5], the best model has been proposed in designing the integration platform by conducting studies on existing integration platform research. Furthermore, research related to data integration platforms has been carried out using ESB, web services, and XML [7,8]. Based on this research, solutions for designing data integration platforms are available by integrating RDMS and application platforms or service integration by utilizing SOA and ESB.

This work introduces UniGate, which serves as a single access point, integrates all information channels, services and applications as a solution. UniGate uses microservice architecture, supports web and mobile platforms. With UniGate, the notification and approval of many systems are concentrated, and users can access to a user manual, frequently asked questions (FAQs), and are supported 24/7 through AI Bot. On the business side, the process of system integration will help information be fully managed, ready for exploitation on big data and management decisions.

The remainder of the thesis is organized as follows. Section 2 describes system requirements. The system design and implementation are presented in section 3. Then, conclusions and perspectives are made in section 4.

Chapter 2

System Architecture and Requirements

2. System Architecture and Requirement

2.1 System Architecture

UniGate is the result of applying and integrating different technologies and architectures for the FPT Software management information system. The architecture of UniGate is described in Figure 1.

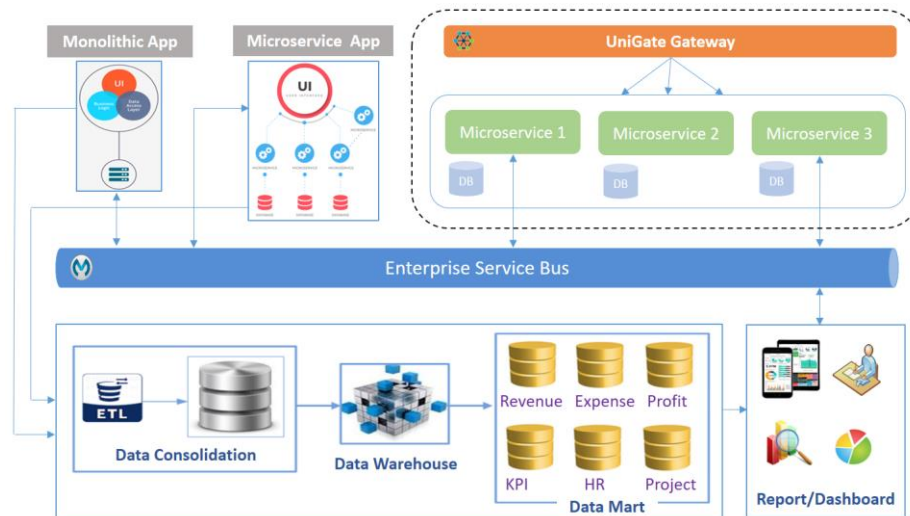


Figure 1. Architecture overview of UniGate System

The UniGate system consists of main components as below:

- **Operation Systems:** Including monolithic applications & microservice applications, which are used for Human Resource, Delivery, Finances, Marketing, etc.
- **Integration System:** Including an infrastructure of enterprise service bus. ESB is used for integrating services in the application landscape, allows different applications to communicate with each other by acting as a transit system for carrying data between applications
- **Data Consolidation:** Combining data information from operation systems in many different formats, unifying and storing it in a raw database. The area sits between the data source(s) and the data target(s), which are often data warehouses, data marts, or other data repositories.
- **Data warehouse/Data Mart:** Storing data from operation systems, used for reporting and analyzing.
- **Report/Dashboard:** Providing analysis and summary reports to support decision-making
- **UniGate application:** unique interface between the user and UniGate, which is integrated with artificial intelligence. Through this interface, users can work with services within the enterprise easily without the need for separate portals or applications for each service.

2.2 System requirements

The system has three main types of user that are system normal users, managers and administrators. The functional requirements of each user are described in Table 1.

Type	Content
Administrator	Create/Manage schedule
	Manage applications: Add new, Edit, Delete
	Manage app status: Disable, Enable
	Manage User Profile: Create, Edit, Remove, Upload Image
	Monitor API: Search API, View API status
	View log: View, Export
Manager	All functions of normal users
	Approval: View all ticket need to Approval
Normal User	Manage application: View, Add/remove favorites, Search
	Notification: View notifications from all systems
	Chatbot: Conversation between users and an AI bot
	Ticket: Log ticket support
	Landing Page: View report/dashboard

Table 1. Functional requirements

Chapter 3

System design and implementation

3. System Design and Implementation

3.1 Servers

The system consists of many servers configured in Table 2.

No	Configuration			Type	Number	Function	Environment
	CPU (Cores)	RAM (Gb)	HDD (Gb)				
1	16	16	500	Virtual	1	Application	TEST
2	16	16	500	Virtual	1	Database	TEST
3	16	32	500	Virtual	1	Application/ Database	UAT
4	16	32	500	Virtual	1	Application/ Database	PRE/PRODUCTION
5	16	32	1024	Virtual	3	Database	PRODUCTION
6	16	32	500	Virtual	1	API	PRODUCTION
7	16	32	1024	Virtual	3	Gateway Microservice	PRODUCTION
8	16	32	1024	Virtual	1	Chatbot	PRODUCTION

Table 2. List of servers used in the system

3.2 Monolithic and Microservice applications

Some applications in the system have all the business components packaged together, distributed and deployed as a whole, this development and deploy pattern is called monolithic [9,10]. To realize the highly available and flexibly, the monolithic application deployed as a whole, and the load balancing carrying out at the front end using a load balancer. When it comes to the monolithic architecture system, we cannot make replicated the relevant component or module for extending, and deploying the entire application across multiple nodes will a cause waste of resources. Other existing applications will be migrated as well as new applications that will use the microservice architecture [9,10]. The microservice architecture is a lightweight, miniaturized development and operation mode. Each microservice deployed independently and follow the single responsibility principle, different microservice can use different technology stack, and update independently.

3.2.1 Monolithic

Monolithic architecture is considered a traditional way to build applications. A monolithic application is built as a single unit and cannot be divided. Typically, such a solution involves client-side user interfaces, server-side applications, and databases. It is unified and all functions are managed and serviced in one place. Typically, monolithic applications have a large code base and lack modularity. If developers want to update or change something, they access the same code base. So they make changes in the entire stack at once.



Figure 2. Monolithic Application

Benefits

- **Less cross-cutting concerns:** Cross-cutting concerns are the concerns that affect the whole application such as logging, handling, caching, and performance monitoring. In a monolithic application, this area of functionality concerns only one application so it is easier to handle it.
- **Easier debugging and testing:** In contrast to the microservices architecture, monolithic applications are much easier to debug and test. Since a monolithic app is a single indivisible unit, you can run end-to-end testing much faster.
- **Simple to deploy:** Another advantage associated with the simplicity of monolithic apps is easier deployment. When it comes to monolithic applications, you do not have to handle many deployments – just one file or directory.
- **Simple to develop:** As long as the monolithic approach is a standard way of building applications, any engineering team has the right knowledge and capabilities to develop a monolithic application.

Drawbacks

- **Understanding:** When a monolithic application scales up, it becomes too complicated to understand. Also, a complex system of code within one application is hard to manage.
- **Making changes:** It is harder to implement changes in such a large and complex application with highly tight coupling. Any code change affects the whole system so it has to be thoroughly coordinated. This makes the overall development process much longer.
- **Scalability:** You cannot scale components independently, only the whole application.
- **New technology barriers:** It is extremely problematic to apply a new technology in a monolithic application because then the entire application has to be rewritten.

3.2.2 Microservice

The microservice architecture has a significant impact on the relationship between the application and the database. Instead of sharing a database with other microservices, each microservice has its own database. It often leads to duplication of some data but having a database on each micro service is essential if you want to benefit from this architecture because it ensures loose coupling. Another advantage of having a separate database for each microservice is that each microservice can use the type of database best suited to its needs.

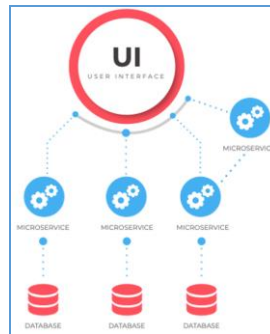


Figure 3. Microservice Application

The principle of microservice

Single responsibility: This is one of the principles defined as part of the SOLID design. It states that a micro unit, class, method, or service should have one and only one responsibility. Each microservice must have a unique responsibility and provide a unique function. You could also say that the number of micro services you should develop is equal to the number of functions you require. The database is also decentralized and in general, each microservice has its own database.

Built around business capabilities: In today's world where so many technologies exist, there's always one that best fits a particular function. But in monolithic applications, it's a big drawback, because we can use different technologies for each function and therefore, need to compromise in specific areas. A micro service will never restrict the application of technology stack or auxiliary database storage to best suit business purposes, i.e. each microservice can use different technologies, based on business requirements.

Design for failure: the microservice must be designed with failure cases in mind. The microservice must take advantage of this architecture and going down a microservice will not affect the entire system, other functions must still be accessible to the user. But this is not the case in Monolithic applications, in which the failure of a module leads to the collapse of the entire application.

Microservices Platforms

Microservice platforms, such as PaaS and Docker scheduler, support the operation and communication of your microservices. These technologies enable communication between microservices for deployment, log analysis, and monitoring.

For example, these platforms support HTTP and REST with load balancing and service discovery. Limited operations support is needed for the implementation of microservices, so they can be quickly deployed and support multiple microservices.

Microservices platforms represent a simplification and solution to common issues. Some notable platforms are Kubernetes and Docker, which is very important for microservices operations. PaaS and Cloud Foundry are also useful but they are not as popular.

It's important to note that migration to these platforms requires a change to the operation and installation of applications, which can make the use of a microservices platform a big and timely step. This is the main drawback of microservice platforms.

Technology stacks and architecture patterns

It is one thing to understand how microarchitecture works and another to actually build and implement it. That's why we want to focus on the various technologies available to you for an entire microservices system. Let's walk through some different tech stacks, patterns, and designs for creating an executable microservice architecture.

Micro and Macro architecture decisions

It is advisable to divide your architecture into micro and macro architecture. The micro architecture involves all decisions made for each microservice. The macro architecture involves all decisions made at a global level that apply to all microservices.

It is possible to extend the concept of micro and macro architecture to technical decisions. Technical decisions can be made within the framework of macro or micro architecture. For example, take a look at the technical decisions to be made at the micro and macro levels for a database:

- **Micro:** Each microservice can have its own instance of the database. If databases were defined at the micro architecture, a crash of one database will only lead to the crash of one microservice. This makes the whole app far more robust.
- **Macro:** The database can also be defined as part of the macro architecture. Multiple microservices must not share a database schema.

Self-contained systems

A self-contained system (SCS) is a type of microservice architecture that specifies the elements of a macro architecture. This means they do not represent the whole system. Since an SCS is self-contained, it provides everything you need to implement one part of the domain logic, such as log data and a UI. SCSs also have an optional API.

For example, an SCS for a microservice payment would store information relevant to that payment as a bounded context. It would also implement the UI to show the payment history, and data about the customers would be replicated from other SCSs.

Think of these as a collection of best practices; SCSs provide precise rules based on established patterns, offering a point of reference for how to build a microservice architecture. All of these rules ensure that an SCS implements a domain, so an added feature only changes one SCS.

We can think of an SCS as a microservice architecture because it can be deployed independently and divide a system into independent web applications. In fact, one SCS can even be split into several microservices. They differ from microservices in three main ways: they are larger than microservices; they focus on loose coupling; they must have a UI.

Frontend Integration

Microservices can also be integrated with a web frontend. Dividing the frontend into different modules helps to solve some of the problems that come from treating it as a monolith. A modularized frontend is made up of separately deployable microservices. This can bring many benefits to your frontend.

For example, a modularized frontend can have independent domain logic, and a change in the domain can be implemented simply by modifying only one microservice. To combine separate frontend, they must be integrated, so an integration system is necessary.

This can be accomplished through links, where one frontend displays a link that another frontend reads and handles. This can also be accomplished through redirects, for example, how OAuth2 handles frontend integration. Redirects combine data transfer with frontend integration.

There are, however, a few exceptions when a frontend should be deployed as a monolith. For example, native mobile applications should be deployment monoliths or if there were a singular team responsible for frontend development.

Asynchronous Microservices

Synchronous microservices make a request to other microservices while it processes requests and waits for results. Asynchronous communication protocols send messages that the recipients react to, but there is no direct response. A microservice could be defined as asynchronous if it does not make requests to other microservices while processing, or it makes requests but does not wait for results.

Asynchronous microservices offer several notable advantages to synchronous microservices and resolve many of the challenges of distributed systems. The logic required to handle microservice requests does not depend on results, making them far more independent.

Similarly, if a communication partner were to fail, it does not crash the whole system, offering overall resilience to your system. On top of that, the processing and delivery are almost always guaranteed.

Some common examples of technologies for asynchronous microservices are Kafka (a MOM commonly used for messaging), REST and Atom data format (for additional infrastructure).

Microservices and Docker

Docker and Microservices are nearly synonymous. Microservices must be separately deployable, scalable independent units. But, what if you create multiple microservices for your application? Docker is a lightweight solution to deploying microservices. A microservice can be packed into a Docker image and isolated as a Docker container. This way, you can build an application that is independent of your host environment.

Instead of having a complete virtual machine of their own, Docker containers share the kernel of the operating system on the Docker host. The processes from the containers appear in the process table of the operating system on which the Docker containers are running.

To use Docker with microservices, you need to create Docker images via files named Dockerfile. Dockerfiles are easy to write, so rolling out software can be easy.

A typical microservice system contains multiple Docker containers. Coordinating a system of multiple Docker containers requires configurations for the virtual network. Containers must be able to find each other in order to communicate. The Docker Compose environment can contact another server via a link, offering a service discovery system.

3.3 UniGate System

In the UniGate system, microservice applications are developed based on JHipster, a development platform to generate, develop and deploy Microservice applications [11].

A typical JHipster application will include three components. First, JHipster Registry is an essential component in the microservices architecture because it connects all the other components and provides the communication between these components. Second, the microservice application which will provide the backend capabilities through exposing the API. Third, the microservices gateway is the frontend of the whole system which will include all the APIs of every microservice application in the system. UniGate's microservice architecture is described in Figure 2.

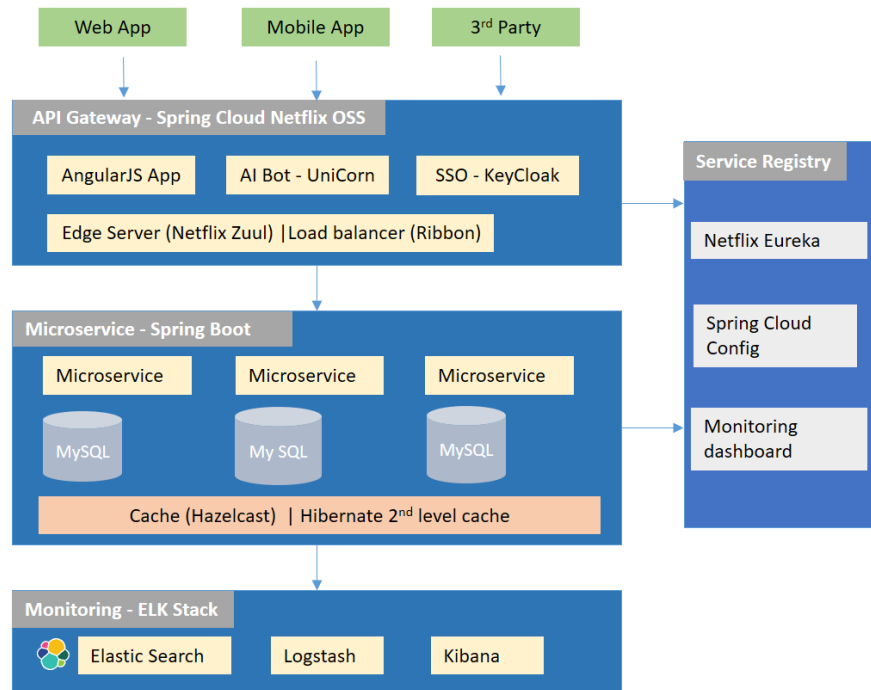


Figure 4. UniGate's microservice architecture

UniGate has an API Gateway that handles web traffic and also acts as a front-end application. It uses several common microservices of available information system (permission, organization, Single Sign On (SSO)) and has three new microservices: *Application Management*, *Notification*, *UniCorn*. All microservices are registered in Service Registry. UniGate uses ELK Stack (Elasticsearch, Logstash, Kibana) [12] for managing and tracking log.

Some of the main features of UniGate are:

- Centralize all notifications
- Integrate SSO using Keycloak, an open-source software product to allow single sign-on with Identity Management and Access Management [13]
- Integrate UniCorn - an AI bot that will be described below
- Plug/Un-plug application, allow users to manage applications, easy to add or remove applications that appear on the list
- Full-text search using Elastic Search for searching user guide, FAQ and other documents

3.4 AI Bot

A chatbot is an artificial intelligence (AI) software that can simulate a conversation (or a chat) with a user in natural language through messaging applications, websites, mobile apps or through the telephone. Chatbot programmed to interact with customers via various text or voice-based interfaces and complete functions ranging from basic (e.g., changing passwords) to complex (e.g., completing banking transactions). The tasks they can handle depends on the goal of the program and the type of bot.

3.4.1 Why are chatbots important

A chatbot is often described as one of the most advanced and promising expressions of interaction between humans and machines. However, from a technological point of view, a chatbot only represents the natural evolution of a Question Answering system leveraging Natural Language Processing (NLP). Formulating responses to questions in natural language is one of the most typical Examples of Natural Language Processing applied in various enterprises' end-use applications.

3.4.2 Types of Conversational Bots

Informational: Informational bots understand natural language and deal with basic questions posed by customers, including initiating a feedback chat or providing answers to generic FAQs to reduce call volume.

Personalized: Integrated with a brand's enterprise system, personalized bots have access to specific customer data, enabling interaction and resolution on a deeply individual level. From troubleshooting Wi-Fi connections, to targeted online shopping offers, personalized bots have proven effective in improving First Contact Resolution (FCR).

Transactional: Pulling data directly from back-end systems, transactional bots execute specific transactions on behalf of customers, such as changing payment methods or account plans. They have the potential to improve operational efficiency of the enterprise while also increasing Customer Satisfaction (CSAT) scores.

3.4.3 Benefits of Conversational Bots

Call deflection: A high number of the contacts that would have come straight to the call center can be handled by Conversational Bots, allowing agents to concentrate on the most complex cases. Recent Gartner research shows that organizations report a reduction of up to 70% in chat and/or email inquiries after implementing chatbot technology.

Voice of the Customer: Bots can capture and analyze vast amounts of customer data and analyze it through Natural Language Processing (NLP) and text analytics to determine patterns of behavior.

Cost reduction: Automating processes previously carried out by agents helps to reduce operational costs in your contact center. According to Juniper Research, enterprises that adopt chatbot technology will realize cost savings of \$11 billion annually by 2023.

Reduce customer effort: By reducing resolution times and unnecessary agent handling, Conversational Bots decrease customer friction to improve brand loyalty and customer lifetime value.

Empower agents: By handling routine customer service interactions, conversational bots free up human agents to focus on higher-value transactions.

3.4.4 RASA Platform

AI Bot that integrates with UniGate is a chatbot which provides support service for all internal user of the FPT Software information system. The chatbot is called UniCorn. It allows users to chat and request many supporting services, offers both live chat and chatbot. The supporter can stop a bot response so he or she can take over the conversation. UniCorn is based on RASA - an open-source AI Bot platform [14].

Rasa is an open source machine learning framework for automated text and voice-based conversations. Understand messages, hold conversations, and connect to messaging channels and APIs.

This diagram shows the basic steps of how an assistant built with Rasa responds to a message:

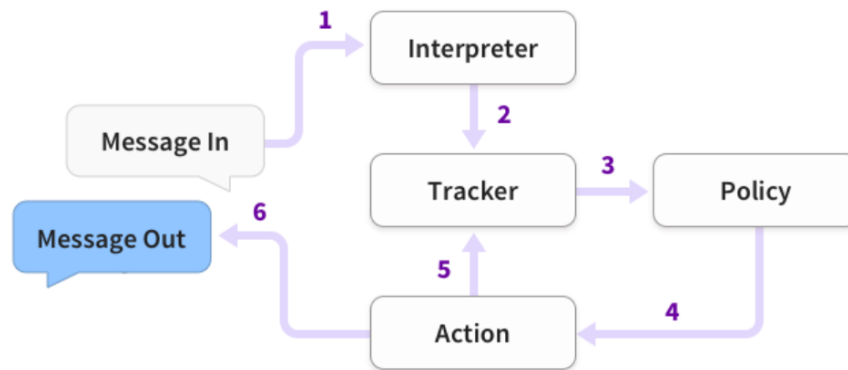


Figure 5. RASA message response

Rasa NLU – Nature Language Understanding for Chatbots and AI assistants

Rasa NLU is important part of RASA. This is an open-source natural language processing tool for intent classification, response retrieval and entity extraction in chatbots. For example, taking a sentence like "I am looking for a Vin restaurant in the center of town" and returning structured data like

```

{
  "intent": "search_restaurant",
  "entities": {
    "cuisine": "Vin",
    "location": "center"
  }
}

```

Training NLU-only models

To train an NLU model only, run:

```

rasa train nlu

```

This will look for NLU training data files in the data/ directory and saves a trained model in the models/ directory. The name of the model will start with nlu-.

Testing NLU model on the command line

To try out NLU model on the command line, use the `rasa shell nlu` command:

```
rasa shell nlu
```

This will start the `rasa shell` and ask you to type in a message to test. You can keep typing in as many messages as you like.

Alternatively, you can leave out the `nlu` argument and pass in an `nlu-only` model directly:

```
rasa shell -m models/nlu-20190515-144445.tar.gz
```

Running an NLU server

To start a server with your NLU model, pass in the model name at runtime:

```
rasa run --enable-api -m models/nlu-20190515-144445.tar.gz
```

You can then request predictions from your model using the `/model/parse` endpoint. To do this, run:

```
curl localhost:5005/model/parse -d '{"text":"hello"}'
```

Training Data Format

Data Formats

You can provide training data as Markdown or as JSON, as a single file or as a directory containing multiple files. Note that Markdown is usually easier to work with.

Markdown Format

Markdown is the easiest Rasa NLU format for humans to read and write. Examples are listed using the unordered list syntax, e.g. minus -, asterisk *, or plus +. Examples are grouped by intent, and entities are annotated as Markdown links, e.g. [`<entity text>`](`<entity name>`), or by using the following syntax [`<entity-text>`]{`"entity": "<entity name>"`}. Using the latter syntax, you can also assign synonyms, roles, or groups to an entity, e.g. [`<entity-text>`]{`"entity": "<entity name>", "role": "<role name>", "group": "<group name>", "value": "<entity synonym>"`}. The keywords `role`, `group`, and `value` are optional in this notation.

The training data for Rasa NLU is structured into different parts:

- common examples

- synonyms

- regex features and

- lookup tables

While common examples is the only part that is mandatory, including the others will help the NLU model learn the domain with fewer examples and also help it be more confident of its predictions.

Synonyms will map extracted entities to the same name, for example mapping “my savings account” to simply “savings”. However, this only happens *after* the entities have been extracted, so you need to provide examples with the synonyms present so that Rasa can learn to pick them up.

Lookup tables may be specified as plain text files containing newline-separated words or phrases. Upon loading the training data, these files are used to generate case-insensitive regex patterns that are added to the regex features.

JSON Format

The JSON format consists of a top-level object called `rasa_nlu_data`, with the keys `common_examples`, `entity_synonyms` and `regex_features`. The most important one is `common_examples`.

```
{
  "rasa_nlu_data": {
    "common_examples": [],
    "regex_features" : [],
    "lookup_tables" : [],
    "entity_synonyms": []
  }
}
```

The `common_examples` are used to train your model. You should put all of your training examples in the `common_examples` array. Regex features are a tool to help the classifier detect entities or intents and improve the performance.

Improving Intent Classification and Entity Recognition

Common Examples

Common examples have three components: text, intent and entities. The first two are strings while the last one is an array.

The text is the user message [required]

The intent is the intent that should be associated with the text [optional]

The entities are specific parts of the text which need to be identified [optional]

Entities are specified with a start and an end value, which together make a range to apply to the string, e.g. in the example below, with `text="show me chinese restaurants"`, then `text[8:15] == 'chinese'`. Entities can span multiple words, and in fact the value field does not have to correspond exactly to the substring in your example. That way you can map synonyms, or misspellings, to the same value.

```
## intent:restaurant_search
```

```
- show me [chinese](cuisine) restaurants
```

Regular Expression Features

Regular expressions can be used to support the intent classification and entity extraction. For example, if your entity has a deterministic structure (like a zipcode or an email address), you can use a regular expression to ease detection of that entity. For the zipcode example it might look like this:

```
## regex:zipcode
```

- [0-9]{5}

regex:greet

- hey[^\s]*

The name doesn't define the entity nor the intent, it is just a human readable description for you to remember what this regex is used for and is the title of the corresponding pattern feature. As you can see in the above example, you can also use the regex features to improve the intent classification performance.

Try to create your regular expressions in a way that they match as few words as possible. E.g. using `hey[^\s]*` instead of `hey.*`, as the later one might match the whole message whereas the first one only matches a single word. Regex features for entity extraction are currently only supported by the `CRFEntityExtractor` component! Hence, other entity extractors, like `MitieEntityExtractor` or `SpacyEntityExtractor` won't use the generated features and their presence will not improve entity recognition for these extractors. Currently, all intent classifiers make use of available regex features.

Lookup Tables

Lookup tables provide a convenient way to supply a list of entity examples. The supplied lookup table files must be in a newline-delimited format.

When lookup tables are supplied in training data, the contents are combined into a large, case-insensitive regex pattern that looks for exact matches in the training examples. These regexes match over multiple tokens, so lettuce wrap would match get me a lettuce wrap ASAP as `[0 0 0 1 1 0]`. These regexes are processed identically to the regular regex patterns directly specified in the training data.

Normalizing Data

Entity Synonyms

If you define entities as having the same value they will be treated as synonyms. Here is an example of that:

intent:search

- in the center of [NYC>{"entity": "city", "value": "New York City"}

- in the centre of [New York City](city)

As you can see, the entity city has the value New York City in both examples, even though the text in the first example states NYC. By defining the value attribute to be different from the value found in the text between start and end index of the entity, you can define a synonym. Whenever the same text will be found, the value will use the synonym instead of the actual text in the message.

To use the synonyms defined in your training data, you need to make sure the pipeline contains the `EntitySynonymMapper` component (see [Components](#)).

Alternatively, you can add an "entity_synonyms" array to define several synonyms to one entity value.

Language Support

You can use Rasa to build assistants in any language you want! Rasa's supervised_embeddings pipeline can be used on training data in any language. This pipeline creates word embeddings from scratch with the data you provide.

Training a Model in Any Language

Rasa's supervised_embeddings pipeline can be used to train models in any language, because it uses your own training data to create custom word embeddings. This means that the vector representation of any specific word will depend on its relationship with the other words in your training data. This customization also means that the pipeline is great for use cases that hinge on domain-specific data, such as those that require picking up on specific product names.

To train a Rasa model in your preferred language, define the supervised_embeddings pipeline as your pipeline in your config.yml or other configuration file.

After you define the supervised_embeddings processing pipeline and generate some NLU training data in your chosen language, train the model with `rasa train nlu`. Once the training is finished, you can test your model's language skills. See how your model interprets different input messages via:

```
rasa shell nlu
```

Choosing a Pipeline

In Rasa, incoming messages are processed by a sequence of components. These components are executed one after another in a so-called processing pipeline defined in your config.yml. Choosing an NLU pipeline allows you to customize your model and finetune it on your dataset

Entity Extraction

Entity extraction involves parsing user messages for required pieces of information. Rasa Open Source provides entity extractors for custom entities as well as pre-trained ones like dates and locations.

3.5 UniCorn Bot

UniCorn architecture is described in Figure 3.

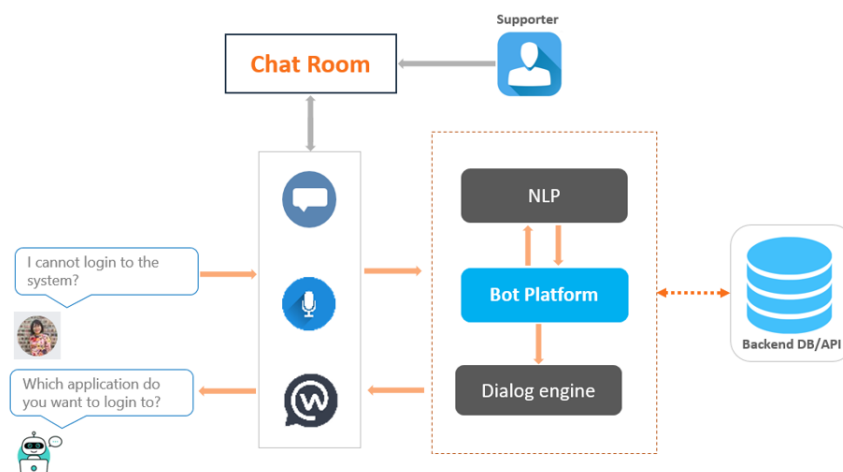


Figure 6. UniCorn Bot architecture

UniCorn has many intelligent features. It is a self-learning bot and be integrated with enterprise apps and legacy systems. UniCorn supports conversation channels across text, voice and custom channels and in multi-lingual. This AI Bot can be fully deployed on both on-premise and cloud environments.

3.6 Enterprise Service Bus

ESB is a standardized integration platform that combines messaging & web service. This platform provides a dependable and scalable infrastructure that connects disparate applications and IT resources, mediates their incompatibilities, orchestrates their interactions, and makes them broadly available as services for additional uses.

3.6.1 ESB and SOA

An ESB is an essential component of SOA, or service-oriented architecture, an architecture that emerged in the late 1990s. SOA defines a way to make software components reusable via service interfaces. These interfaces utilize common communication standards in such a way that they can be rapidly incorporated into new applications without having to perform deep integration each time.

Each service in an SOA embodies the code and data integrations required to execute a complete, discrete business function (e.g. checking a customer's credit, calculating a monthly loan payment, or processing a mortgage application). The service interfaces provide loose coupling, meaning they can be called with little or no knowledge of how the integration is implemented underneath. The services are exposed using standard network protocols—such as SOAP (simple object access protocol)/HTTP or JSON/HTTP—to send requests to read or change data. The services are published in a way that enables developers to quickly find them and reuse them to assemble new applications.

These services can be built from scratch but are often created by exposing functions from legacy systems of record as service interfaces. This is where the need for an ESB arises. Legacy systems and systems of record typically use old protocols and proprietary data formats that need to be translated and integrated to work with the SOA network protocols. An ESB performs these translations and integrations on the fly. You could implement an SOA without an ESB, but application owners would each have to find their own unique way to expose service interfaces, which is a lot of work (even if the interfaces are eventually reusable) and creates a significant maintenance challenge in the future.

3.6.2 Benefits

A centralized ESB offers the potential to standardize—and dramatically simplify—communication and integration of services across the enterprise. Hardware and software costs can be shared, provisioning the servers need only be performed once, and a single team of specialists can be tasked (and, if necessary, trained) to develop and maintain the integrations.

Developers can use a single protocol to 'talk' to the ESB and issue commands that direct interactions between services and leave it to the ESB to translate the commands, route the messages, and transform the data as required to get the commands executed. This would enable developers to spend dramatically less time integrating and much more time configuring and improving their applications. And the ability to reuse these integrations from one project to the next offered the potential for still greater productivity gains and savings downstream.

But while ESBs were deployed successfully in many organizations, in many other organizations the ESB came to be seen as the bottleneck in the SOA deployment. Making changes or enhancements to one integration often destabilized others. Updates to the ESB middleware often impacted existing integrations. Because the ESB was centrally managed, applications teams soon found themselves waiting in line for their integrations. As the volume of integrations grew, implementing high availability and disaster recovery for the ESB servers became more costly. And as a cross-enterprise project, the ESB proved difficult to fund, making these technical challenges that much more difficult to resolve.

Ultimately the challenges of maintaining, updating, and scaling a centralized ESB proved so overwhelming and expensive that the ESB often delayed the very productivity gains that it, and the SOA, were intended to yield, frustrating line of business teams who anticipated a greater pace of innovation.

3.6.3 ESB vs. Microservices

Microservices architecture enables the internals of a single application to be broken up into small pieces that can be independently changed, scaled, and administered. Microservices emerged and gained steam with the rise of virtualization, cloud computing, Agile development practices, and DevOps. In these contexts, microservices offer the following:

- **Improved developer agility and productivity** by enabling developers to incorporate new technologies into one part of an application without touching or ‘catching up’ the rest of the application.
- **Simpler, more cost-effective scalability** by enabling any component to be scaled independently of others, for the fastest possible response to workload demands and the most efficient use of computing resources.
- **Greater resilience**, because the failure of one component does not impact the others, and each microservice can perform to its own availability requirements without staking the other components to a ‘greatest common availability’ requirement.

The same granularity that microservices bring to application design can be brought to integration, with similar benefits. This is the idea behind agile integration, which breaks up the ESB into fine-grained, decentralized integration components, without interdependencies, that individual application teams can own and manage themselves.

When Should an ESB be Used?

Are there best practices to determine when the implementation of an ESB is worthwhile? The use of an ESB is worth considering when three or more applications or services need to be integrated. A simple point-to-point integration is significantly easier and much more cost-effective when connecting two applications. An ESB can also be worthwhile if services are going to be incorporated from external service providers over which the company has no control. The ESB can then be used to monitor the service level agreements that the external provider guarantees. The impact of the adjustments to service contracts can further be kept to a minimum, as the ESB continues to provide a stable interface while making the necessary changes to the messages.

If many different protocols, such as HTTP, SOAP, and FTP, are to be used and standardized to one protocol like SOAP, the ESB can perform the necessary protocol transformation. If services are to be consistently incorporated into an architecture to be able to receive, process, and produce messages, then the use of ESB is also suitable. This is also applicable if a collection of pre-

defined components and adapters need to be accessed, which allows various protocols and legacy applications to be integrated in a standardized fashion. If messages need to be reliably and securely processed, the ESB can simplify the implementation of transactional message flows between two heterogeneous transactional data sources.

Using an ESB can become problematic if large volumes of data need to be sent via the bus as a large number of individual messages. ESBs should never replace traditional data integration like ETL tools. Data replication from one database to another can be resolved more efficiently using data integration, as it would only burden the ESB unnecessarily.

An ESB should support stateless message flows if long-term business processes are to be implemented. Long-running business processes are stateful and can best be implemented using BPEL and/or BPMN. These are not usually available via the ESB, but rather via a business process management system (BPMS).

ESB Blueprint

Due to the lack of standardization, the ESB market is rather confusing. There are many products that claim to be ESBs but offer quite different solutions and are based on different architectures. To allow for more effective evaluation of ESB products, the various functions that are assigned to an ESB have been arranged into a blueprint (Figure 2).

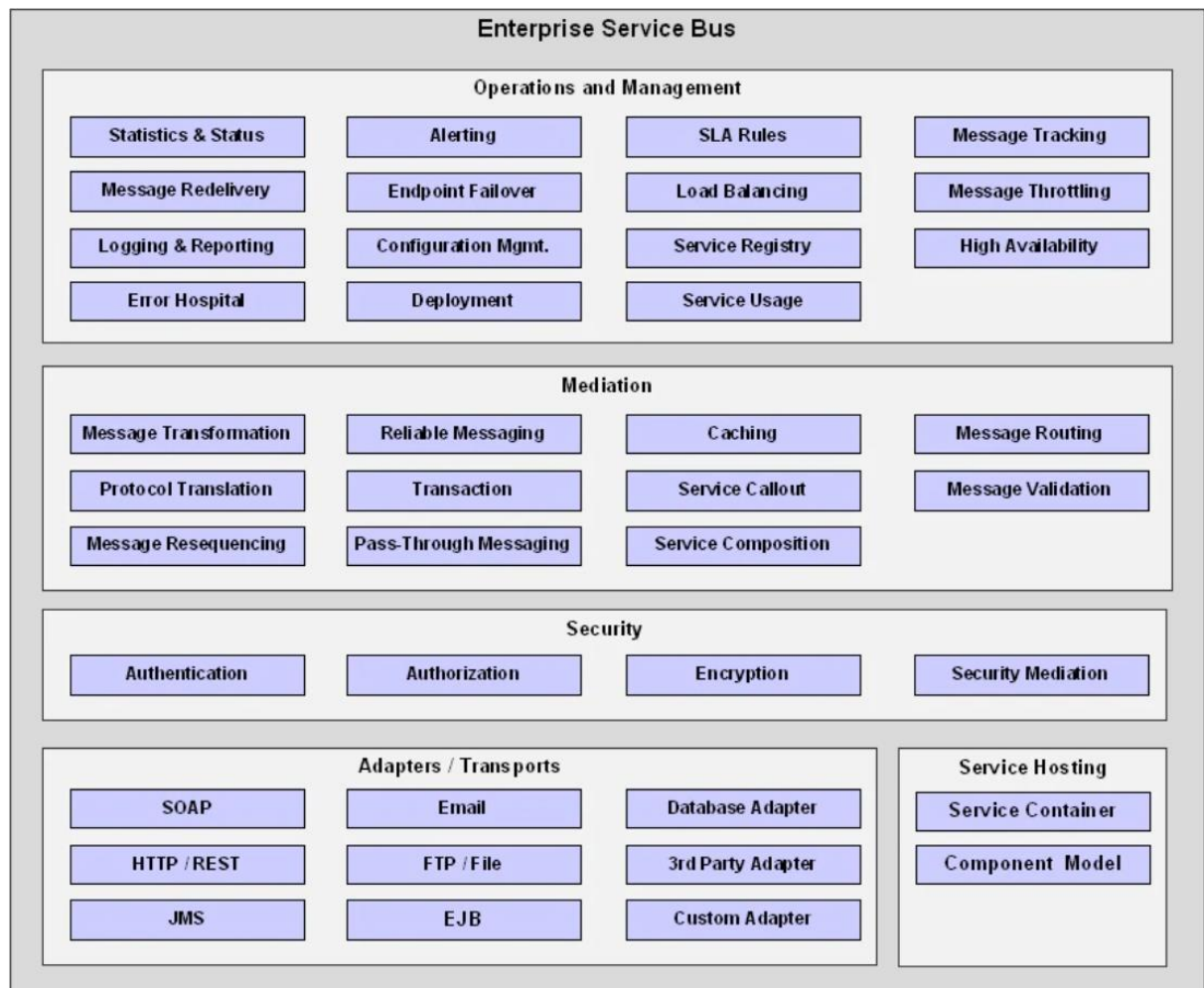


Figure 7. The blueprint for an enterprise service bus

The ESB blueprint diagram doesn't include an "orchestration" or "process choreography" component, as it is considered to be part of the BPMS category. These offer dedicated runtime environments for long-running, stateful business processes that are optimized for this and support languages such as BPMN or BPEL. ESBs should be stateless and configured to process messages as efficiently as possible.

Operations and Management Module

The following functional components in this module enable reliable operation and management of the enterprise service bus. Statistics & Status provides the services' ESB statistics, such as their number of errors, minimum and maximum response times, and number of processed messages. Alerting offers a mechanism for sending alert messages that can be sent via various channels so that existing monitoring environments can also be incorporated. SLA Rules are rules that can be defined on the basis of information from the Statistics & Status functional component. This allows SLAs to be measured and monitored. Any SLA infringements are notified using the Alerting component.

Message Tracking provides the option of easily tracking messages within the ESB, and should be activated whenever required so as to minimize any associated overhead. Message Redelivery ensures that messages that aren't processed immediately are automatically resent after a pre-defined period of time. The number of attempts and the interval between them can be configured. This component is primarily suitable for technical errors that only last a certain length of time, such as temporary network outages. Endpoint Failover enables the option of specifying an alternate service provider that is automatically called whenever the primary service provider is not available.

Load Balancing allows several service endpoints to be listed for one logical service provider endpoint. It uses redundant service implementations that are called alternately for each request according to a defined strategy, which can be round-robin or according to message priority or load dependency.

Message Throttling makes it possible to define a maximum number of messages per unit of time for a service endpoint that should be sent to the service provider. It prevents the service provider from being overloaded at peak times by buffering messages that lie over the threshold in a queue in the ESB. Message Throttling can also support message priorities so that messages of higher priority are always processed first. Logging & Reporting allows messages to be logged and then easily displayed at a later time. It can also provide functional auditing.

Configuration Management enables secure configuration adjustments to the ESB on an operational system, while constantly upholding the integrity of the configuration. Artifacts and attributes can be adapted and replaced during operation. A history of the changes can also be kept so that an ESB service can be rolled back to an earlier status at any time. Service Registry offers the option of registering and managing services on the ESB. High Availability ensures that the services provided by the ESB are failsafe, regardless of the status of the server on which it is operated.

The Error Hospital is the destination for the messages that can't be processed after multiple redelivery attempts, where they can be viewed, corrected if necessary, and reprocessed. Deployment offers the option of installing services automatically on the ESB. Environment-specific parameters such as endpoint URIs are typically overridable by this component. Service Usage allows the use of services to be logged and charged to the user.

Mediation Module

The mediation module contains the functional components that are used to implement the message flow of an ESB service. Message Routing allows messages to be forwarded to a particular service endpoint depending on their content. The criteria for forwarding may originate from the message body or the message header, if the protocol used or the message format supports a header area that is independent of the message body.

Routing based on headers can be an attractive option to improve service performance and scalability, as direct access to the header is more efficient than parsing the routing information from the body. This is of particular consequence for larger messages.

Message Transformation enables conversion from one message format to another that is applicable to text and binary messages as well as XML formats. In addition, there is also the option of converting from text, such as the CSV format, to XML and vice versa. XML transformations use the well-known standard XSLT, which enables declarative descriptions of transformations and has graphical resources with drag-and-drop functionality for creation purposes.

A major drawback of XSLT transformations is a high memory usage if large documents are being processed, which may restrict the scalability of a solution. It is preferable if the ESB offers transformation options that support XML streaming, such as via XQuery.

Service Callout offers the option of accessing other services within a message flow in the ESB, such as to enhance a message. A service may be a Web service but the ESB can conceivably enable program code that's installed locally on the ESB to be called directly, such as a Java class method. Reliable Messaging is the support of reliable message transfer using queuing or WS* standards, such as WS-ReliableMessaging.

Protocol Translation means the possibility of switching from a certain communication protocol to a different one without any programming effort, such as from TCP/IP to HTTP. Message Validation ensure that messages are valid. In the case of XML, this means that the message contains well-defined XML and corresponds to a certain XML schema or WSDL. However, the ESB can also offer other validation means, such as Schematron or a rules engine.

Message Exchange Pattern (MEP) is the support of message exchange patterns, such as synchronous and asynchronous request/reply, one-way call, and publish/subscribe. Result Cache provides the option of saving results from a service call in a cache, so that each subsequent call returning the same result can be answered from the cache without calling the service again. This particularly applicable if the data is static or requires sporadic or infrequent changes. Potentially expensive operations, such as accessing a legacy system, can be reduced significantly.

Transaction allows ESBs offer transactional integrity through message processing. The persistent queues that the ESB uses to support Reliable Messaging generally work as transactional data sources, and can therefore participate in heterogeneous transactions. In addition, the ESB can offer a distributed transaction manager that can coordinate distributed transactions via heterogeneous data sources using the two-phase commit protocol.

Message Resequencing allows a flow of messages that belong together but aren't in the correct order to be resequenced. In a message-oriented solution with parallel processing of messages, the sequence in which the messages enter the ESB can be lost. Message Resequencing can be incorporated into the message flow If the sequence is of import for the service provider. A

resequencer contains an internal buffer that processes the messages until the complete sequence is available and can be sent.

Pass-Through Messaging provides efficient forwarding of messages by the ESB. This is useful if the ESB is to be used for service virtualization and the messages are forwarded from the service consumer to the service provider unchanged. In this case, it is applicable if the ESB doesn't touch the message and simply passes it on as is.

Security Module

This module supports both the transport-level and message-level security using a number of components. Authentication authenticates service consumers when they access the service in the ESB and verifies ESB authentication for the service provider. Authorization provides an authorization system for services that can often be configured via XACML to be assigned to users or roles.

Security Mediation provides support for interactions that communicate outside of security domains by converting credentials from one domain to the corresponding credentials of the other domain. Encryption/Decryption supports the encryption and decryption of the content of a message.

Adapters/Transport Module

This module includes adapters for connecting services that are provided by the ESB via the Service Hosting module. The ESB can be assumed to provide a set of adapters from the ground up, and also has the option for customers or third-party developers to develop additional adapters for customer-specific requirements.

Service Hosting Module

This module allows services to be installed and operated directly on the ESB and is usually required if the ESB is based on an application server. Service Container provides one or more containers in which the services are installed and service lifecycles managed. It offers the service access to technical cross-sectional functions, such as transactions and security.

The Component Model provides an abstract component model, such as Java EJB, Java Spring Framework, or Microsoft COM+, on the basis on which the services are created.

3.6.4 Mule ESB

Mule ESB [15] (community version) is used as the integration platform in the FPT Software information system as Figure 4.

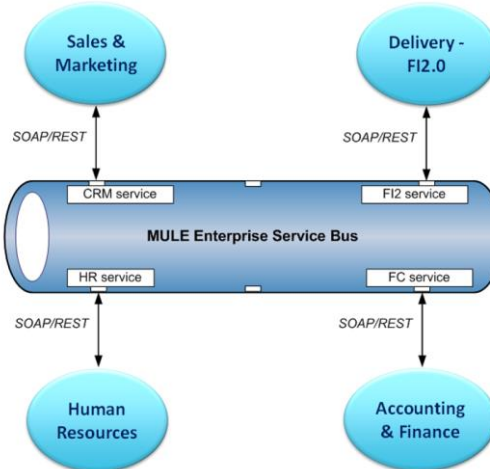


Figure 8. Integration platform (ESB)

3.7 Data Consolidation & Data Warehouse

Data consolidation is the process that combines all data wherever it may live, removes any redundancies, and cleans up any errors before it gets stored in a data warehouse.

3.7.1 Data Consolidation

How to consolidate data

Data consolidation isn't standard across industries or organizations and there are a few different tools or methods you can use to do it:

Hand-coding or scripting - This manual process custom builds scripting by data scientists to combine and consolidate data from a predetermined range of sources.

Open-source tools - Open-source software helps organizations combine and consolidate data with relatively little cost and more flexibility, but requires a higher degree of expertise in coding and usually more manpower.

Cloud-based tools - A modern approach to data consolidation, cloud-based tools automate many data consolidation tasks with speed, scalability, and security.

Challenges with data consolidation

Even though data consolidation is a critical stepping-stone on the path to greater business intelligence and faster, more precise decisions, it isn't always realistic for organizations to do it themselves using existing teams and systems. On the upside, this more traditional approach may give the impression that your organization has full control of its data. On the downside, it can introduce a slew of other challenges that cancel out any control you may believe you have.

Here are four common roadblocks that can occur with traditional, on-site data consolidation:

Limited time - IT teams already have their hands full configuring, maintaining, and monitoring on-site hardware and other equipment, in addition to keeping up with the rest of their daily tasks. So spending the necessary hours to script, run, and manage error-free data consolidation may not always be feasible for your current team.

Limited resources - Any data integration process usually requires the help of skilled data scientists. Yet many organizations don't have the budget or internal buy-in to staff up with the

right resources to get the job done. The truth is, acquiring specialized knowledge is time-consuming, and hiring it is a hefty investment.

Scattered locations - Many businesses operate with remote or branch locations, which means that data isn't available in a single physical place but has to be secured and managed in multiple locations. When you need to retrieve that outlying data and combine it with local data sources, it can take significantly more time (and a lot more bandwidth). Yet time is not a friend when quick decisions are on the line, since data can fast become outdated.

Security issues - Every place where data is stored opens up the potential for a hack or breach. And moving data to another place during the data consolidation process only increases that potential. As well, most businesses have to adhere to some level of regulatory standards. But patched equipment and having just one systems admin in charge of data management for the entire enterprise makes it much more difficult to maintain security and compliance to the degree necessary.

Modern cloud-based data consolidation

Where businesses can gain an advantage, however, is by using third-party, cloud-based data consolidation tools. Like other cloud solutions, these tools are built for speed, security, scalability, and flexibility — no matter where or in what form your data exists. In this way, you're assured a complete and accurate data load you can then access whenever you want in your data repository of choice.

And because it's all happening in the cloud, you don't have to purchase or maintain expensive equipment to handle the data consolidation aspect, or add resources to your in-house IT team to oversee a manual process.

Alooma's modern data consolidation solution ticks all the boxes your organization would need to kickstart a full data integration strategy. With security and compliance a top priority, Alooma's cloud-based solution moves and combines all your data following the highest industry standards, thereby reducing risk.

Fast and efficient data pipelines cleanse your data and make it available to you in real time rather than in batches, if you choose. And because the data consolidation process is automated, your data operations run much smoother and more efficiently all around, saving upfront infrastructure costs and improving the speed and soundness of business decisions.

3.7.2 Oracle Data Integrator

Oracle Data Integrator [16] is used to extract all data from FPT Software operation systems, transform and load it into a centralized raw database.

The Business Problem

In today's increasingly fast-paced business environment, organizations need to use more specialized software applications; they also need to ensure the coexistence of these applications on heterogeneous hardware platforms and systems and guarantee the ability to share data between applications and systems. Projects that implement these integration requirements need to be delivered on-spec, on-time and on-budget.

A Unique Solution

Oracle Data Integrator employs a powerful declarative design approach to data integration, which separates the declarative rules from the implementation details. Oracle Data Integrator is

also based on a unique E-LT (Extract - Load Transform) architecture which eliminates the need for a standalone ETL server and proprietary engine, and instead leverages the inherent power of your RDBMS engines. This combination provides the greatest productivity for both development and maintenance, and the highest performance for the execution of data transformation and validation processes.

Here are the key reasons why companies choose Oracle Data Integrator for their data integration needs:

- **Faster and simpler development and maintenance:** The declarative rules driven approach to data integration greatly reduces the learning curve of the product and increases developer productivity while facilitating ongoing maintenance. This approach separates the definition of the processes from their actual implementation, and separates the declarative rules (the "what") from the data flows (the "how").
- **Data quality firewall:** Oracle Data Integrator ensures that faulty data is automatically detected and recycled before insertion in the target application. This is performed without the need for programming, following the data integrity rules and constraints defined both on the target application and in Oracle Data Integrator.
- **Better execution performance:** traditional data integration software (ETL) is based on proprietary engines that perform data transformations row by row, thus limiting performance. By implementing an E-LT architecture, based on your existing RDBMS engines and SQL, you are capable of executing data transformations on the target server at a set-based level, giving you much higher performance.
- **Simpler and more efficient architecture:** the E-LT architecture removes the need for an ETL Server sitting between the sources and the target server. It utilizes the source and target servers to perform complex transformations, most of which happen in batch mode when the server is not busy processing end-user queries.
- **Platform Independence:** Oracle Data Integrator supports all platforms, hardware and OSs with the same software.
- **Data Connectivity:** Oracle Data Integrator supports all RDBMSs including all leading Data Warehousing platforms such as Oracle, Exadata, Teradata, IBM DB2, Netezza, Sybase IQ and numerous other technologies such as flat files, ERPs, LDAP, XML.
- **Cost-savings:** the elimination of the ETL Server and ETL engine reduces both the initial hardware and software acquisition and maintenance costs. The reduced learning curve and increased developer productivity significantly reduce the overall labor costs of the project, as well as the cost of ongoing enhancements.

ODI Component Architecture

The Oracle Data Integrator platform integrates in the broader Fusion Middleware platform and becomes a key component of this stack. Oracle Data Integrator provides its run-time components as Java EE applications, enhanced to fully leverage the capabilities of the Oracle WebLogic Application Server. Oracle Data Integrator components include exclusive features for Enterprise-Scale Deployments, high availability, scalability, and hardened security.

Repositories

The central component of the architecture is the Oracle Data Integrator Repository. It stores configuration information about the IT infrastructure, metadata of all applications, projects, scenarios, and the execution logs. Many instances of the repository can coexist in the IT infrastructure, for example *Development*, *QA*, *User Acceptance*, and *Production*. The

architecture of the repository is designed to allow several separated environments that exchange metadata and scenarios (for example: Development, Test, Maintenance and Production environments). The repository also acts as a version control system where objects are archived and assigned a version number.

The Oracle Data Integrator Repository is composed of one *Master Repository* and several *Work Repositories*. Objects developed or configured through the user interfaces are stored in one of these repository types.

There is usually only one master repository that stores the following information:

- Security information including users, profiles and rights for the ODI platform
- Topology information including technologies, server definitions, schemas, contexts, languages and so forth.
- Versioned and archived objects.

The work repository is the one that contains actual developed objects. Several work repositories may coexist in the same ODI installation (for example, to have separate environments or to match a particular versioning life cycle). A Work Repository stores information for:

- Models, including schema definition, data stores structures and metadata, fields and columns definitions, data quality constraints, cross references, data lineage and so forth.
- Projects, including business rules, packages, procedures, folders, Knowledge Modules, variables and so forth.
- Scenario execution, including scenarios, scheduling information and logs.

When the Work Repository contains only the execution information (typically for production purposes), it is then called an Execution Repository.

ODI Studio and User Interfaces

Administrators, Developers and Operators use the Oracle Data Integrator Studio to access the repositories. This Fusion Client Platform (FCP) based UI is used for administering the infrastructure (security and topology), reverse-engineering the metadata, developing projects, scheduling, operating and monitoring executions.

ODI Studio provides four Navigators for managing the different aspects and steps of an ODI integration project:

- *Designer Navigator* is used to design data integrity checks and to build transformations such as for example:
 - Automatic reverse-engineering of existing applications or databases
 - Graphical development and maintenance of transformation and integration interfaces
 - Visualization of data flows in the interfaces
 - Automatic documentation generation
 - Customization of the generated code
- *Operator Navigator* is the production management and monitoring tool. It is designed for IT production operators. Through Operator Navigator, you can manage your interface executions in the sessions, as well as the scenarios in production.
- *Topology Navigator* is used to manage the data describing the information system's physical and logical architecture. Through Topology Navigator you can manage the topology of your information system, the technologies and their datatypes, the data servers linked to these technologies and the schemas they contain, the contexts, the

languages and the agents, as well as the repositories. The site, machine, and data server descriptions will enable Oracle Data Integrator to execute the same integration interfaces in different physical environments.

- *Security Navigator* is the tool for managing the security information in Oracle Data Integrator. Through Security Navigator you can create users and profiles and assign user rights for methods (edit, delete, etc) on generic objects (data server, datatypes, etc), and fine-tune these rights on the object instances (Server 1, Server 2, and so forth).

Oracle Data Integrator also provides a Java API for performing all these run-time and design-time operations. This *Oracle Data Integrator Software Development Kit (SDK)* is available for standalone Java applications and application servers.

Run-Time Agent

At design time, developers generate scenarios from the business rules that they have designed. The code of these scenarios is then retrieved from the repository by the Run-Time Agent. This agent then connects to the data servers and orchestrates the code execution on these servers. It retrieves the return codes and messages for the execution, as well as additional logging information – such as the number of processed records, execution time and so forth - in the Repository. The Agent comes in two different flavors:

- The Java EE Agent can be deployed as a web application and benefit from the features of an application server.
- The Standalone Agent runs in a simple Java Machine and can be deployed where needed to perform the integration flows.

Both these agents are multi-threaded java programs that support load balancing and can be distributed across the information system. This agent holds its own execution schedule which can be defined in Oracle Data Integrator, and can also be called from an external scheduler. It can also be invoked from a Java API or a web service interface.

Oracle Data Integrator Console

Business users (as well as developers, administrators and operators), can have read access to the repository, perform topology configuration and production operations through a web based UI called *Oracle Data Integrator Console*. This web application can be deployed in a Java EE application server such as Oracle WebLogic.

To manage and monitor the Java EE and Standalone Agents as well as the ODI Console, Oracle Data Integrator provides a new plug-in that integrates in Oracle Fusion Middleware Control Console.

3.7.3 Data Warehouse

Data warehouse is a subject-oriented, integrated, nonvolatile, time-variant data set for decision making support in enterprise management. Data warehouse can cross time and space boundaries of the enterprise, integrating the data of different companies and different processing systems to provide a unified view, which can access more comprehensive information about company operation and customer behavior.

Types of Data Warehouse

Enterprise Data Warehouse

Enterprise Data Warehouse is a centralized warehouse. It provides decision support service across the enterprise. It offers a unified approach for organizing and representing data. It also provide the ability to classify data according to the subject and give access according to those divisions.

Operational Data Store

Operational Data Store, which is also called ODS, are nothing but data store required when neither Data warehouse nor OLTP systems support organizations reporting needs. In ODS, Data warehouse is refreshed in real time. Hence, it is widely preferred for routine activities like storing records of the Employees.

Data Mart

A data mart is a subset of the data warehouse. It specially designed for a particular line of business, such as sales, finance, sales or finance. In an independent data mart, data can collect directly from sources.

General stages of Data Warehouse

Earlier, organizations started relatively simple use of data warehousing. However, over time, more sophisticated use of data warehousing begun.

The following are general stages of use of the data warehouse:

Offline Operational Database

In this stage, data is just copied from an operational system to another server. In this way, loading, processing, and reporting of the copied data do not impact the operational system's performance.

Offline Data Warehouse

Data in the Datawarehouse is regularly updated from the Operational Database. The data in Datawarehouse is mapped and transformed to meet the Datawarehouse objectives.

Real time Data Warehouse

In this stage, Data warehouses are updated whenever any transaction takes place in operational database. For example, Airline or railway booking system.

Integrated Data Warehouse

In this stage, Data Warehouses are updated continuously when the operational system performs a transaction. The Datawarehouse then generates transactions which are passed back to the operational system.

Data Warehouse components

Load manager: Load manager is also called the front component. It performs with all the operations associated with the extraction and load of data into the warehouse. These operations include transformations to prepare the data for entering into the Data warehouse.

Warehouse Manager: Warehouse manager performs operations associated with the management of the data in the warehouse. It performs operations like analysis of data to ensure

consistency, creation of indexes and views, generation of denormalization and aggregations, transformation and merging of source data and archiving and baking-up data.

Query Manager: Query manager is also known as backend component. It performs all the operation operations related to the management of user queries. The operations of this Data warehouse components are direct queries to the appropriate tables for scheduling the execution of queries.

End-user access tools: This is categorized into five different groups like 1. Data Reporting 2. Query Tools 3. Application development tools 4. EIS tools, 5. OLAP tools and data mining tools.

Data Warehouse Implementation

The best way to address the business risk associated with a Datawarehouse implementation is to employ a three-prong strategy as below

Enterprise strategy: Here we identify technical including current architecture and tools. We also identify facts, dimensions, and attributes. Data mapping and transformation is also passed.

Phased delivery: Data warehouse implementation should be phased based on subject areas. Related business entities like booking and billing should be first implemented and then integrated with each other.

Iterative Prototyping: Rather than a big bang approach to implementation, the Datawarehouse should be developed and tested iteratively.

Microsoft SQL Server [17] is used to implement the Data Warehouse solution for the FPT Software. In this solution, the online analytical processing (OLAP) tool is an integral part. It based on the data warehouse and can perform the on-line data access and analysis for specific questions. The company used SQL Server Business Intelligence Development Studio (BIDS) [18] for this task. The goal is to meet the decision-making support or the particular query and reporting demand under the multi-dimensional environment. A data mart is a subset of a data warehouse oriented to a specific business line. Data marts contain repositories of summarized data collected for analysis on a specific section or unit within the organization, some data mart have been built are Revenue, Expense, Profit, KPI, Project, etc by Month, Quarter, Year.

3.8 Web Application

The main screen of the Web application is in Figure 5. There are some functions as below:

- List all applications by category
- Ticket approve
- Web push notifications
- Chat bot

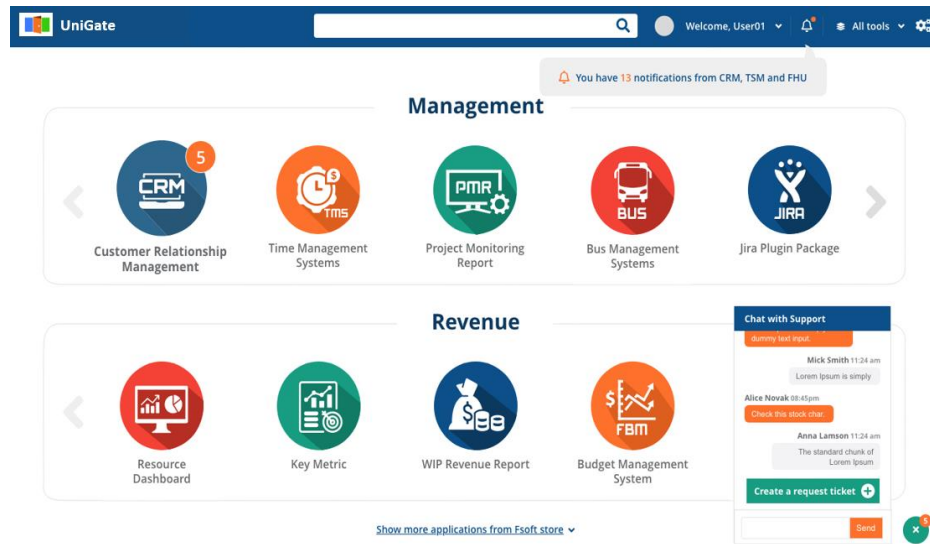


Figure 9. UniGate main screen

Chapter 4 Result

4. Result

About 50 applications have been integrated with the Unigate with more than 4,500 users visit and use regularly. Unicorns are supplemented with up to 300 skills with more than 950 short conversations, saving user support time up to 30%. Unigate has also gathered 34,590 notifications from different applications, in additional more than 100 tickets created on unigate each month. Unigate is also used as a training channel for new employee about information systems.

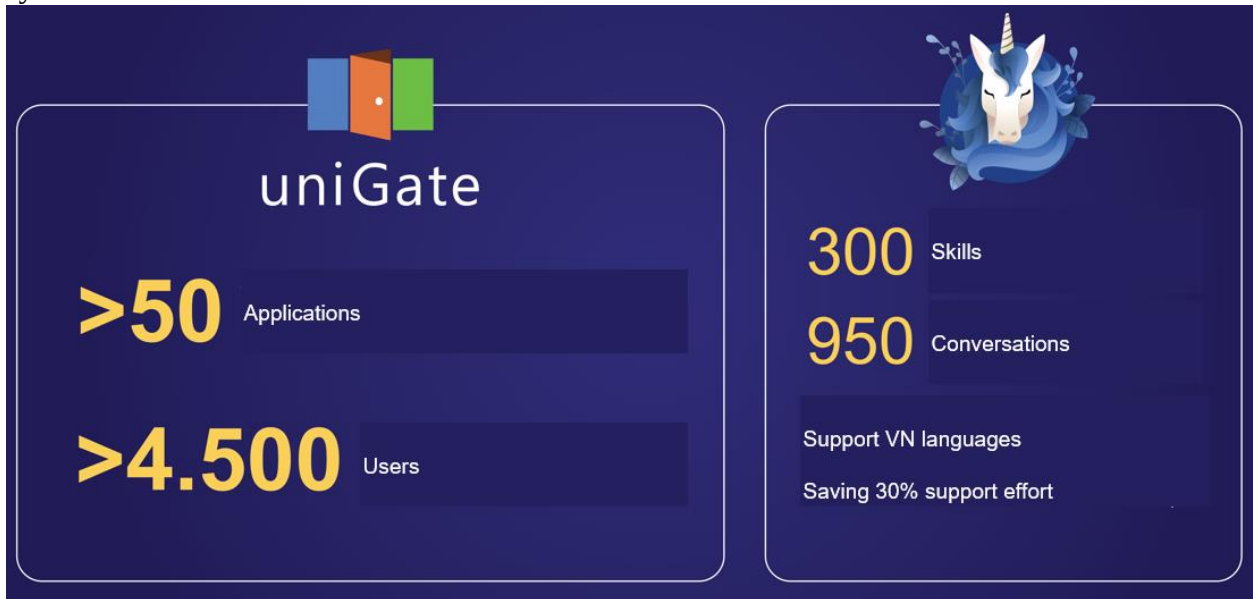


Figure 10. UniGate statistics

Figures from 11 to 14 show some statistics of the amount of traffic and user interaction with the system: the number of users accessing the system, the total number of users accessing over time, and the location of employees who are using the system worldwide.

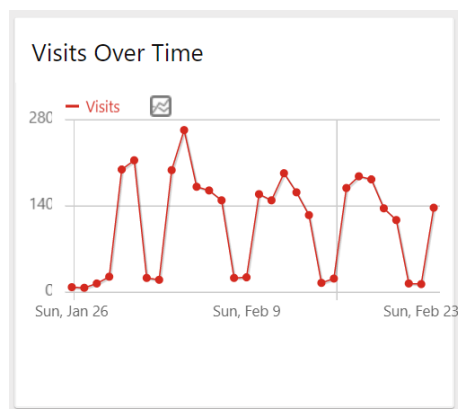


Figure 11. Visitor Over Time

DATE	VISITS	ACTIONS
Last 24 hours	153	202
Last 30 minutes	11	20

Figure 12. Visitor in Real-Time

Visitor Map



Figure 13. Visitor Map

Browsers

BROWSER		UNIQUE VISITORS
	Chrome	90
	Safari	4
	Chrome Mobile	4
	Mobile Safari	2

Figure 14. Visitor Browsers

Chapter 5

Conclusion and Perspective

5. Conclusion and Perspective

The implementation of a centralized and intelligent access point for large organizations is essential. The thesis presents the information system integration solution for the FPT software company in Vietnam. The work describes the architectures and technologies applied to build an intelligent access point that brings whole new experiences to all employees.

Micro-service applications are based on the JHipster framework. Future integration of the framework with automated source generation is expected to accelerate application development by 30%. In addition, JHipster also can be integrated with the ability to test performance so that applications can be verified their own load capacity before releasing products.

Unicorn not only works independently on a separate unigate channel, but also integrates with popular chat channels in the enterprise such as Teams, Workchat. Unicorn also needs to expand the range of support for languages other than Vietnamese such as English, Japanese, which are two relatively popular languages used within the Company. Along with that, Unicorn will also be upgraded to become an AI Voice Chat assistant through integration with Mozilla DeepSpeech and Mozilla TTS.

Data processing is being done through batch processing, which limits the system's ability to integrate real time, to overcome this limitation, the system will be supplemented with streaming processing, support real time and near real time via Kafka Streaming platform.

In the near future, the Unigate will be integrated with the forum using Xenforo platform, this is a place to discuss and support information systems and is widely deployed for the Corporation.

This solution is also fully applicable for similar company models and is also a good reference for research directions of Software engineering, Information System, etc.

References

- [1] FPT Software, <https://www.fpt-software.com/about-fpt-software/>. Accessed 2020/02/01.
- [2] Hahn, M., Saez, S.G., Andrikopoulos, V., et al.: Development and evaluation of a multi-tenant service middleware PaaS solution. In: *Proceedings of the IEEE/ACM 7th International Conference on Utility and Cloud Computing*, London, pp. 278-287. doi: 10.1109/UCC.2014.37 (2014)
- [3] Llambías, G., Ruggia, R.: A middleware-based platform for the integration of bioinformatic services. In: *Proceedings of the XL Latin American Computing Conference (CLEI)*, Montevideo, pp. 1-12. doi: 10.1109/CLEI.2014.6965178 (2014)
- [4] Delgado, A., González, L., Ruggia, R.: A Process-Aware Inter-organizational Service Integration Platform to Support Collaborative Organizations. In: *Proceedings of the IEEE International Conference on Services Computing (SCC)*, San Francisco, CA, pp. 844-847. doi: 10.1109/SCC.2016.120 (2016)
- [5] Singh, P.M., Van, S.M., Wieringa, R.: Reference Architecture for Integration Platforms. In: *Proceedings of the IEEE 21st International Enterprise Distributed Object Computing Conference (EDOC)*, Quebec City, QC, pp. 113-122. doi: 10.1109/EDOC.2017.24 (2017)
- [6] Priantari, R., Kurniawan, N.B.: Integrated IT Service Platform: Systematic Literature Review and Meta-Analysis. In: *Proceedings of the International Conference on Information Technology Systems and Innovation (ICITSI)*, Bandung - Padang, Indonesia, pp. 562-567. doi: 10.1109/ICITSI.2018.8696013 (2018)
- [7] Fan, J., Xu, C., Xiong, L.: Data transformation of ESB based on the data model for integration. In: *Proceedings of the 9th International Conference on Fuzzy Systems and Knowledge Discovery*, Sichuan, pp. 2645-2651. doi: 10.1109/FSKD.2012.6233914 (2012)

- [8] Qiu, D., Liu, J., Zhao, G.: Design and application of data integration platform based on web services and XML. In: Proceedings of the 6th International Conference on Electronics Information and Emergency Communication (ICEIEC), Beijing, pp. 253-256. doi: 10.1109/ICEIEC.2016.7589732 (2016)
- [9] Furda, A., Fidge, C., Zimmermann, O., et al.: Migrating Enterprise Legacy Source Code to Microservices: On Multitenancy, Statefulness, and Data Consistency. *IEEE Software*, vol. 35, no. 03, pp. 63-72, 2018. doi: 10.1109/MS.2017.440134612 (2018)
- [10] Velepucha, V., Flores, P., Torres, J.: Migration of Monolithic Applications Towards Microservices Under the Vision of the Information Hiding Principle: A Systematic Mapping Study. In: Botto-Tobar M., León-Acurio J., Díaz Cadena A., Montiel Díaz P. (eds) *Advances in Emerging Trends and Technologies*. ICAETT 2019. *Advances in Intelligent Systems and Computing*, vol 1066. Springer, Cham (2020)
- [11] <https://www.jhipster.tech/>. Accessed 2020/02/01.
- [12] <https://www.elastic.co/what-is/elk-stack>. Accessed 2020/02/01.
- [13] <https://www.keycloak.org/>. Accessed 2020/02/01.
- [14] <https://rasa.com/>. Accessed 2020/02/01.
- [15] <https://www.mulesoft.com/platform/soa/mule-esb-open-source-esb>. Accessed 2020/02/01.
- [16] <https://www.oracle.com/middleware/technologies/data-integrator.html>. Accessed 2020/02/01.
- [17] <https://www.microsoft.com/en-us/sql-server/sql-server-downloads>. Accessed 2020/02/01.
- [18] <https://www.microsoft.com/en-us/sql-server/sql-business-intelligence>. Accessed 2020/02/01.
- [19] <https://www.educative.io/blog/microservices-architecture-tutorial-all-you-need-to-get-started>
- [20] <https://www.n-ix.com/microservices-vs-monolith-which-architecture-best-choice-your-business/>
- [21] <https://www.geeksforgeeks.org/monolithic-vs-microservices-architecture/>
- [22] <https://expertsystem.com/chatbot/>
- [23] <https://www.oracle.com/technical-resources/articles/middleware/soa-ind-soa-esb.html>
- [24] <https://www.ibm.com/cloud/learn/esb>
- [25] <https://www.alooma.com/blog/what-is-data-consolidation>
- [26] <https://www.guru99.com/data-warehousing.html>
- [27] https://docs.oracle.com/cd/E17904_01/integrate.1111/e12641/overview.htm